A Side-Channel Resistant FPGA Implementation of CRYSTALS-Kyber Using Duplication and Clock Randomization

Michail Moraitis[1], Yanning Ji[1], Martin Brisfors[1], Elena Dubrova[1], Niklas Lindskog[2], and Håkan Englund[2]

[1]Department of Electrical Engineering, Royal Institute of Technology (KTH), Stockholm, Sweden, {micmor, yanning, brisfors, dubrova}@kth.se
[2]Ericsson Research, Ericsson AB, Lund, Sweden

Abstract. CRYSTALS-Kyber has been selected by the NIST as a post-quantum public-key encryption and key establishment algorithm to be standardized. This makes it important to develop side-channel attack resistant implementations of CRYSTALS-Kyber. In this paper, we propose utilizing duplication combined with clock randomization as a means of protecting CRYSTALS-Kyber FPGA implementations from side-channel attacks. Such a countermeasure has been proven effective in ensuring side-channel resistance of AES FPGA implementations. It has the benefits of universal coverage, glitch immunity, and zero clock cycle overhead. We present a protected version of CRYSTALS-Kyber built on the top of the lightweight unprotected implementation by Xing el al. Our security evaluation shows that the protected implementation is resistant to deep learning-based side-channel attacks.

Keywords: CRYSTALS-Kyber, side-channel attack, countermeasure, clock randomization, duplication, deep learning.

1 Introduction

The security of public-key cryptographic schemes employed today relies on the intractability of certain mathematical problems such as integer factorization, or the discrete logarithm. However, if large-scale quantum computers become available, it will be possible to solve these problems in polynomial time using Shor's algorithm [1]. Thus, current public-key cryptographic schemes will be compromised.

To address this issue, in 2016 the National Institute of Standards and Technology (NIST) initiated a standardization process for post-quantum cryptographic primitives, NIST PQC. Currently, the PQC process is in the fourth round. The security of candidate primitives that are evaluated in the PQC process relies on problems believed to be difficult for quantum computers, such as lattice problems and error-correcting code decoding. One of the finalists, IND-CCA2 secure public key encryption (PKE) and key encapsulation mechanism

(KEM) CRYSTALS-Kyber [2], is already selected for standardization [3]. It is also included in the National Security Agency (NSA) suite of cryptographic algorithms recommended for national security systems [4].

This makes it important to design CRYSTALS-Kyber's implementations that are resistant to side-channel attacks. Side-channel attacks exploit information obtained from physically measurable, non-primary channels such as timing, power consumption, or electromagnetic emissions of a device running the implementation. Pioneered by Paul Kocher in 1999 [5], side-channel attacks are considered a main security threat to cryptographic implementations at present.

Both unprotected and protected software implementations of CRYSTALS-Kyber have been already thoroughly analyzed. It has been shown that deep learning-based side-channel attacks can overcome countermeasures such as masking, shuffling, and the combination of both [6]. Attacks on an unprotected hardware implementation of CRYSTALS-Kyber from [7] have also been demonstrated in [8] and [9]. Recently, protected hardware implementations of CRYSTALS-Kyber have been presented in [10] and [11]. The former uses first-order masking and hiding, while the latter uses random delay insertion as well as address and instruction randomization to offer protection against side-channel attacks.

Our contributions: We present a method for protecting field-programmable gate array (FPGA) implementations of CRYSTALS-Kyber from side-channel attacks based on duplication and clock randomization[1]. The protected implementation consists of two cryptographic cores, a primary and a dummy. Each core is controlled by a different randomized clock. Both cores take the same input data but use a different secret and public key pair for operation. In this paper, we focus on Kyber768 but other versions can be protected similarly.

The idea to use duplication combined with a randomized clock as a countermeasure against side-channel analysis (which could be considered a hiding countermeasure) was first proposed in [13] in the context of block ciphers, and demonstrated on the Advanced Encryption Standard (AES). Our results show that while AES (block cipher) and CRYSTALS-Kyber (post-quantum KEM) are very different cryptographic algorithms, this countermeasure is applicable to both. The main contribution of this paper is the security analysis of the resulting protected implementation of CRYSTALS-Kyber. The analysis shows that the combination of duplication and clock randomization provides sufficient protection against side-channel attacks. It also shows that clock randomization alone is not secure.

The duplication and clock randomization countermeasure imposes 100% area overhead. However, it does not increase the number of clock cycles. In contrast, masking penalties both these parameters. Other advantages of duplication and clock randomization countermeasure include:

1. *Simplicity of implementation.* The only module that needs to be designed is the randomized clock generator.

[1] This report is an extended version of [12].

2. *Application independence.* The countermeasure considers the implementation of the algorithm under protection as a black box. Therefore it can be applied to any algorithm/implementation.
3. *Universality of coverage.* It covers the entire implementation, while some countermeasures only cover portions that are believed to be vulnerable. For instance, the key generation algorithm is not covered in the masked implementation of CRYSTALS-Kyber from [10]. Although there have not been any documented attacks on this part, things could change in the future. Since duplication and clock randomization countermeasure is not linked to any specific leakage model or vulnerability, it can potentially provide protection from unanticipated attacks.
4. *Glitch immunity.* Glitches are a common concern in hardware masking schemes. Clock randomization mitigates this problem.
5. *Stronger resistance to repetition attacks.* Since random masks are refreshed at each execution, errors in repeated measurements from a masked implementation are more independent than the errors from an implementation protected using duplication and clock randomization (with fixed keys). This gives an attacker a higher chance to increase the success rate of an attack on a masked implementation by repetitions.

Since nothing comes for free, the duplication and clock randomization countermeasure also has some limitations. One is 100% area overhead caused by duplication. Another one is non-constant throughput and latency caused by the randomized clock[2].

The rest of the paper is organized as follows. Section 2 reviews the previous work. Section 3 gives a background on CRYSTALS-Kyber algorithm. Section 4 describes the presented protected implementation of CRYSTALS-Kyber. Section 5 presents the equipment used in the experiments and the target implementation. Sections 6 and 7 describe the profiling and attack stages, respectively. Section 8 summarises experimental results. Finally, Section 9 concludes the paper.

2 Previous Work

This section describes previous work on unprotected and protected hardware implementations of CRYSTALS-Kyber and related side-channel attacks and countermeasures.

2.1 Implementations

Huang et al. [14] presented an unprotected hardware implementation of CRYSTALS-Kyber in FPGA that takes 68,815 clock cycles to execute the decapsulation procedure of Kyber512. This is 10 times fewer clock cycles than the ARM Cortex-M4 implementation of CRYSTALS-Kyber from [15].

[2] Note that in an infinite time window the average throughput is constant.

Xing el al. [7] developed an even smaller and faster unprotected hardware implementation of CRYSTALS-Kyber, intended for resource-constrained devices. With suitably designed pipelines and highly-optimized architecture, the implementation is capable of executing the decapsulation procedure of any version of Kyber within 14,000 clock cycles.

In [10], a first-order masked hardware implementation of Kyber512 is presented. The masking method adds 70% overall area overhead and 100% clock cycle counts overhead on the top of the unprotected implementation from [14]. The area is traded for time by reusing resources and pipelining. Note that, since the implementation of CRYSTALS-Kyber from [7] already makes full use of pipelining, it would be difficult to mask it with only 70% area overhead and 100% performance overhead. A less secure version of Kyber512, protected with hiding, that has a 60% area overhead and 80% clock cycle count overhead is also proposed in [10].

In [11], a hardware implementation of CRYSTALS-Kyber protected from side-channel analysis by random delay insertion as well as address and instruction shuffling is presented. This side-channel countermeasure adds only 5% area overhead at the expense of a performance overhead. Information on performance overhead is not available in [11].

2.2 Attacks

In [16] a preliminary power analysis of a hardware implementation of CRYSTALS-Kyber from [14] is presented. Some leakage points are discovered using a t-test for the Kyber512 implementation in Xilinx Virtex-7 FPGA.

In [9], a non-profiling correlation analysis-based side-channel attack of Kyber512 implemented in a Xilinx Artix 7 FPGA targeting polynomial multiplication during the PKE decryption step of the decapsulation algorithm is presented. The attack utilizes 166,620 traces, captured with a near-field electromagnetic probe, to recover the secret key. However, the attack requires knowledge of the register reference values. This is typically not possible in a real attack scenario.

In [8], a profiling side-channel attack on a Xilinx Artix-7 FPGA implementation of Kyber768 from [7] is presented. Using deep learning-based power analysis and enumeration, the attack can recover a message of CRYSTALS-Kyber from 5120 traces. In CRYSTALS-Kyber KEM, a message recovery from a properly generated ciphertext implies the shared key recovery since the shared key is derived from the message using hash functions. Furthermore, by recovering messages contained in chosen ciphertext constructed using known methods, e.g. [6], one can extract the secret key of CRYSTALS-Kyber.

2.3 Countermeasures

Clock randomization The idea of using execution time randomization as a countermeasure against side-channel attacks is as the side-channel attacks themselves [17]. Over the years, many techniques have been proposed to implement it, including random delay insertion, dummy operation addition, random execution

order, random branching, and clock randomization. For hardware implementations, random delay addition and clock randomization schemes have received the most attention, e.g. [18–30]. The resistance of these countermeasures to Correlation Power Analysis (CPA) and, more recently, deep-learning (DL)-based side-channel analysis [29] has been evaluated. The evaluations were positive, e.g. [28] and [29] have shown that a CPA using 4M traces and a Convolutional Neural Network (CNN)-based side-channel analysis using 1M traces, respectively, cannot break an FPGA implementation of AES protected by a randomized clock.

However, in [31] it has been demonstrated that clock randomization can be defeated if side-channels are acquired using a sampling frequency considerably higher than the design's clock frequency. The resulting high-resolution measurements can be synchronized with sufficient precision to enable the extraction of the secret key. The authors of [31] demonstrated correlation power analysis (CPA)-based and deep learning-based attacks on an FPGA implementation of AES protected by a randomized clock.

Duplication Duplication is a fault-tolerant technique used for detecting random faults. In hardware security, it is employed as a countermeasure against fault injection. However, duplication alone does not provide adequate protection against side-channel attacks. For example, in [31] is shown that the secret key can be extracted from an FPGA implementation of a duplicated AES that uses different keys in each block.

Masking Masking [32] aims to eliminate the leakage by partitioning a secret variable into two or more shares and executing all operations separately on the shares. Since the shares are randomized at each execution, in theory, they should not contain any exploitable information about the secret variable. However, in practice, even higher-order masked software implementations of CRYSTALS-Kyber have been broken by deep learning-based side-channel analysis [33].

Implementing masking in hardware is known to be a difficult task. Glitches are a common problem in hardware masking schemes such as threshold implementations, domain-oriented masking, unified masking approach, and generic low latency masking. Care should be taken to check whether every possible transition is masked [34].

Shuffling Shuffling [35] is another well-known countermeasure applicable to operations on secret variables that are not dependent on each other. It typically uses the Fisher-Yates algorithm to create a random permutation which is then utilized as the loop iterator to index the processing of the variables inside the loop. By shuffling the order of the operations, the correlation between the executed instructions and time can be hidden. Combined with masking, shuffling was previously believed to provide sufficient protection against side-channel attacks. However, an attack on a software implementation of CRYSTALS-Kyber presented in [6] has shown that the combined protection can be defeated as well.

KYBER.CPAPKE.KeyGen()

1: $(\rho, \sigma) \leftarrow \mathcal{U}(\{0,1\}^{256})$
2: $\mathbf{A} \leftarrow \mathcal{U}(R_q^{k \times k}; \rho)$
3: $\mathbf{s}, \mathbf{e} \leftarrow \beta_{\eta_1}(R_q^{k \times 1}; \sigma)$
4: $\mathbf{t} = \mathsf{Encode}_{12}(\mathbf{As} + \mathbf{e})$
5: $\mathbf{s} = \mathsf{Encode}_{12}(\mathbf{s})$
6: **return** $(pk = (\mathbf{t}, \rho), sk = \mathbf{s})$

KYBER.CPAPKE.Dec$(\mathbf{s}, c)$

1: $\mathbf{u} = \mathsf{Decompress}_q(\mathsf{Decode}_{d_u}(c_1), d_u)$
2: $v = \mathsf{Decompress}_q(\mathsf{Decode}_{d_v}(c_2), d_v)$
3: $\mathbf{s} = \mathsf{Decode}_{12}(\mathbf{s})$
4: $m = \mathsf{Encode}_1(\mathsf{Compress}_q(v - \mathbf{s}^T\mathbf{u}, 1))$
5: **return** m

KYBER.CPAPKE.Enc$(pk = (\mathbf{t}, \rho), m, r)$

1: $\mathbf{t} = \mathsf{Decode}_{12}(\mathbf{t})$
2: $\mathbf{A} \leftarrow \mathcal{U}(R_q^{k \times k}; \rho)$
3: $\mathbf{r} \leftarrow \beta_{\eta_1}(R_q^{k \times 1}; r)$
4: $\mathbf{e}_1 \leftarrow \beta_{\eta_2}(R_q^{k \times 1}; r); e_2 \leftarrow \beta_{\eta_2}(R_q^{1 \times 1}; r)$
5: $\mathbf{u} = \mathbf{A}^T\mathbf{r} + \mathbf{e}_1$
6: $v = \mathbf{t}^T\mathbf{r} + e_2 + \mathsf{Decompress}_q(m, 1)$
7: $c_1 = \mathsf{Encode}_{d_u}(\mathsf{Compress}_q(\mathbf{u}, d_u)$
8: $c_2 = \mathsf{Encode}_{d_v}(\mathsf{Compress}_q(v, d_v)$
9: **return** $c = (c_1, c_2)$

Fig. 1. Description of KYBER.CPAPKE algorithm from [2].

KYBER.CCAKEM.KeyGen()

1: $z \leftarrow \mathcal{U}(\{0,1\}^{256})$
2: $(pk, \mathbf{s}) = $ KYBER.CPAPKE.KeyGen()
3: $sk = (\mathbf{s}, pk, \mathcal{H}(pk), z)$
4: **return** (pk, sk)

KYBER.CCAKEM.Encaps(pk)

1: $m \leftarrow \mathcal{U}(\{0,1\}^{256})$
2: $m = \mathcal{H}(m)$
3: $(\hat{K}, r) = \mathcal{G}(m, \mathcal{H}(pk))$
4: $c = $ KYBER.CPAPKE.Enc(pk, m, r)
5: $K = \mathsf{KDF}(\hat{K}, \mathcal{H}(c))$
6: **return** (c, K)

KYBER.CCAKEM.Decaps
$(sk = (\mathbf{s}, pk, \mathcal{H}(pk), z)$,c)

1: $m' = $ KYBER.CPAPKE.Dec$(\mathbf{s}, c)$
2: $(\hat{K}', r') = \mathcal{G}(m', \mathcal{H}(pk))$
3: $c' = $ KYBER.CPAPKE.Enc(pk, m', r')
4: **if** $c = c'$ **then**
5: **return** $K = \mathsf{KDF}(\hat{K}, \mathcal{H}(c))$
6: **else**
7: **return** $K = \mathsf{KDF}(z, \mathcal{H}(c))$
8: **end if**

Fig. 2. Description of KYBER.CCAKEM algorithm from [2].

3 CRYSTALS-Kyber Algorithm

CRYSTALS-Kyber [2] is an IND-CCA2-secure cryptographic algorithm whose security relies on the hardness of the Module Learning with Errors (M-LWE) problem. An IND-CCA2-secure algorithm is indistinguishable under an adaptive chosen ciphertext attack in theory. However, in practice, the theoretical resistance to chosen-ciphertext attacks can be bypassed with a side-channel analysis [6, 8, 9, 36].

CRYSTALS-Kyber consists of a CPA-secure PKE scheme, KYBER.CPAPKE, and a CCA-secure KEM scheme, KYBER.CCAKEM, based on a post-quantum version of the Fujisaki-Okamoto transform [37]. These algorithms are described in Fig. 1 and Fig. 2 respectively. We follow the notation of [8].

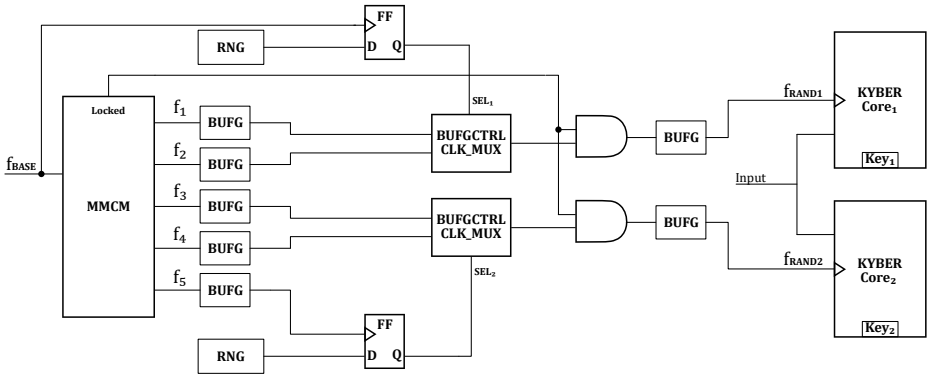

Fig. 3. A block diagram of the protected CRYSTALS-Kyber implementation.

Let $\mathbb{Z}_q$ denote the ring of integers modulo a prime q, and R_q denote the quotient ring $\mathbb{Z}_q[X]/(X^n + 1)$, for $n = 256$ and $q = 3329$. CRYSTALS-Kyber performs computations on vectors of ring elements in R_q^k, where k is the rank of the module used to scale the security levels, $k \in \{2, 3, 4\}$.

There are three different versions of CRYSTALS-Kyber, called Kyber512, Kyber768 and Kyber1024. The security level is determined by the rank of the module k. In this paper, we focus on the version with the rank $k = 3$, Kyber768. Other versions can be handled similarly.

4 Protected Implementation

The block diagram of the presented protected implementation of CRYSTALS-Kyber is shown in Fig. 3. The configuration consists of a randomized clock generator and two CRYSTALS-Kyber cores, a primary, and a dummy. Each core is controlled by a different randomized clock.

4.1 Randomized Clock Generator

Architecture The randomized clock generator consists of a mixed-mode clock manager (MMCM) module, a random number generators (RNG), two 2-to-1 clock multiplexers (BUFCTRL), seven clock buffers (BUFG), two flip-flops (FFs), and two AND gates (one or two LUT6).

The MMCM module takes a base frequency, f_{BASE}, as input and generates five new frequencies, f_1, f_2, f_3, f_4, f_5 as output. The first four frequencies are connected in pairs through clock buffers to the 2-to-1 clock multiplexers.

The select signals, SEL_1 and SEL_2, of the multiplexers are the registered outputs of an RNG. The registers of SEL_1 are clocked with the base frequency, f_{BASE} while the registers of SEL_2 are clocked with the fifth frequency generated from the MMCM, f_5. Furthermore, the registers of SEL_1 and SEL_2 are receiving

different random values from the RNG. The RNG can be either a pseudo-random number generator (PRNG) or a true random number generator (TRNG).

The two resulting randomized clocks, f_{RAND1} and f_{RAND2}, are combined with the *Locked* signal of the MMCM through AND gates to ensure that the MMCM output frequencies are properly generated. Finally, each randomized clock is given to its respective CRYSTALS-Kyber core through a clock buffer.

Functionality In the protected AES implementation presented in [13], the randomized clock generator selects asynchronously among four base frequencies using a LUT-based 4-to-1 MUX. This results in clock pulses of varying frequency and duty cycle. The simulation results of [13] show that at least 403 different frequencies can be generated in this way. However, a downside of the asynchronous selection method is that clock pulses can be too short for the protected circuit to operate correctly. In the experiments of [13], 3% of encryptions produced wrong ciphertexts.

In the method used in this paper, each randomized clock is generated by selecting between two base frequencies in a synchronous, glitch-free manner. To achieve that, instead of a LUT-based MUX, we use BUFGCTRL, a Xilinx clock buffer primitive for a glitchless selection between two signals. Although only two different frequencies are generated by each randomized clock, which is far fewer than the 403 frequencies of [13], there are no errors in the calculations. Our experimental results show that such a weak randomized clock generator provides sufficient protection against side-channel attacks when combined with duplication.

The goal is to have f_{RAND1} and f_{RAND2} overlap as little as possible. For that reason, we have different candidate frequencies for each core and different random values at a different clock frequency for the selection between those candidate frequencies. Regarding the RNG, it is left to the designer to decide if a TRNG is needed or a PRNG is sufficient. The only requirement is that the RNG is able to give a random output per each clock cycle of f_{BASE} and f_5.

4.2 Duplicated CRYSTALS-Kyber Core

In the presented protected implementation, the primary core, KYBER Core_1, and the dummy core, KYBER Core_2, receive the same input data. However, they are programmed with two different long-term secret keys: the real Key_1, and the dummy, Key_2, respectively.

By using a dummy secret key for the duplicated core, we assure that no additional leakage related to Key_1 is created during the execution. Furthermore, if Key_1 is updated, Key_2 should be updated as well. The links between the keys and the inputs given to both cores are necessary to assure that the noise generated by the duplicated core is algorithmic, rather than random. The two keys have to be updated at the same time, since if the dummy key, Key_2 is changed for every execution, the attacker can filter out the segments of traces generated by the duplicated core in the same way as random noise is filtered

out, i.e. by averaging many repeated measurements for the same input. A similar problem occurs is the attacker can control the input given to the duplicated core.

5 Experimental Setup

This section describes the equipment and the FPGA implementations of CRYSTALS-Kyber used in the experiments.

5.1 Target implementations

In the experiments, we consider three different implementations of CRYSTALS-Kyber:

1. The unprotected implementation of Kyber768 from [7], K_u. It is used as a baseline.
2. A protected version of K_u using only clock randomization, K_r.
3. A protected version of K_u using both duplication and clock randomization, K_{dr}.

5.2 Equipment

For trace acquisition, we use a Chipwhisperer-lite board [38] paired with an Artix-7 XC7A100T-2FTG256 FPGA mounted on a CW305 target board. We use the same FPGA device for the profiling and attack stages because an attack that has failed on the profiling board is unlikely to succeed on another board.

The implementations K_u, K_r and K_{dr}, adapted to the CW305 target board, are converted into the configuration bitstreams using Vivado design suite with the default synthesis options. The bitstreams are then loaded into the FPGA.

Note that Vivado design suite may synthesize, place and route the primary and dummy cores differently. For this reason, the area overhead of duplication may not be exactly 100%.

6 Profiling stage

This section presents our profiling strategy.

6.1 Trace acquisition

At the profiling stage, for each implementation K_u, K_r and K_{dr}, we capture from the FPGA a set of power traces for profiling, $\boldsymbol{T}_P$, during the execution of the decapsulation algorithm KYBER.CCAKEM.Decaps. The traces in $\boldsymbol{T}_P$ are captured for input ciphertexts c generated using CPAPKE.Enc for messages m selected at random.

Figs. 4 and 5 (top) show trace segments which we record for K_r and K_{dr}, respectively. These segments represent the encoding of polynomial coefficients

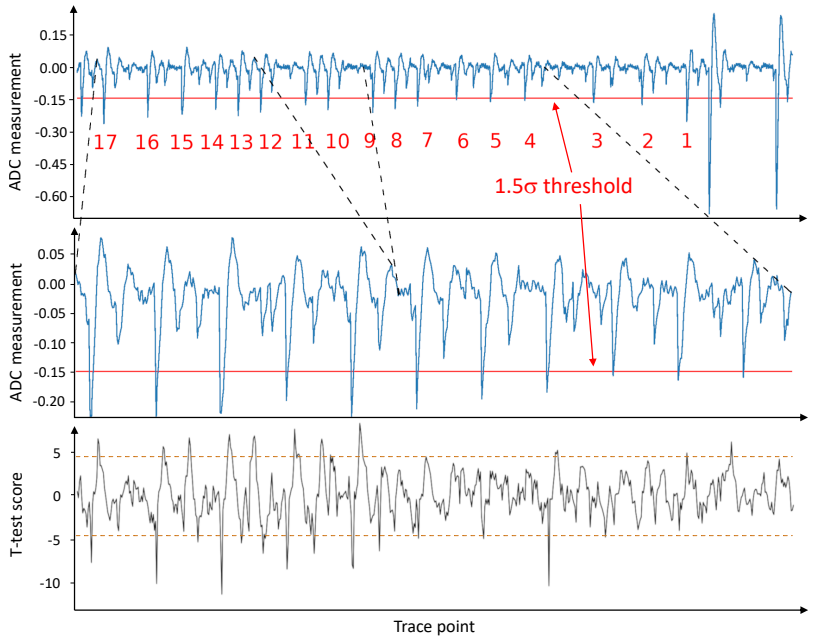

Fig. 4. (Top) A power trace of K_r with $\times 30$ oversampling. The red numbers enumerate the peaks; (middle) the synchronized segments of the top trace which are given as input to neural networks; (bottom) t-test scores for message byte 24 on 200K synchronized traces. The dashed lines indicate the -4.5, 4.5 margins.

into message bits during the decryption step of the decapsulation algorithm. The trace segments in Figs. 4 and 5 contain the processing of one message byte, byte 24. This byte is processed last.

One can see that the peaks in in Figs. 4 and 5 are desynchronized due to the randomized clock.

6.2 Trace synchronization

To synchronize the traces captured from K_r and K_{dr}, we use the synchronization method of [31] adapted to the specific of CRYSTALS-Kyber algorithm.

The method of [31] is applied to traces captured from a protected by a randomized clock FPGA implementation of AES which computes one round per clock cycle. Thus, in the attack on AES, only a single clock cycle needs to be synchronized. However, in the attack on CRYSTALS-Kyber, one needs to synchronize multiple clock cycles because the leakage is distributed among multiple points.

First, we search for an easily identifiable starting point for counting clock cycles. In the attack of AES [31], this point can be either the beginning or the end of the algorithm. However, this is not feasible in the attack on CRYSTALS-Kyber

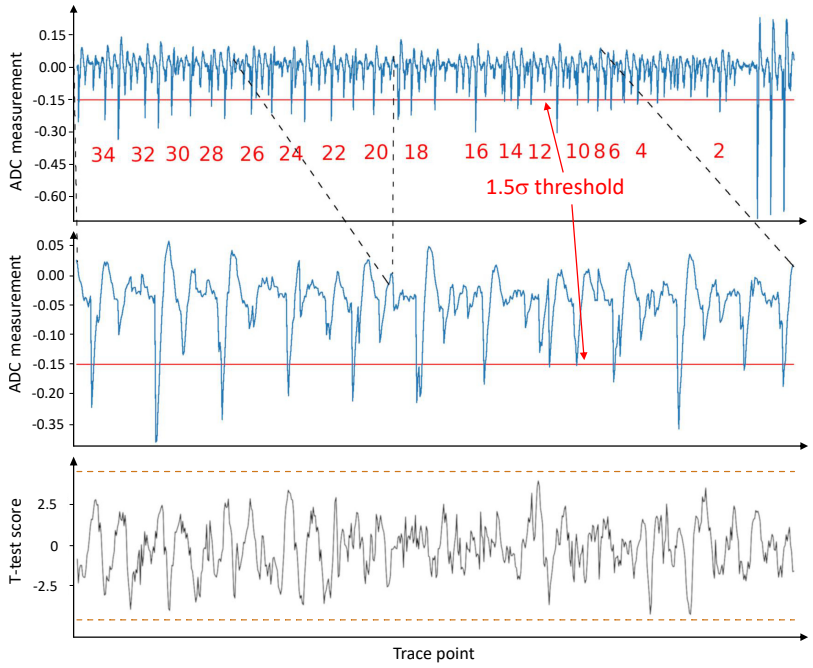

Fig. 5. (Top) A power trace of K_{dr}. The red numbers enumerate the peaks; (middle) the synchronized segments of the top trace which are given as input to neural networks; (bottom) t-test scores for message byte 24 on 200K synchronized traces.The dashed lines indicate the -4.5, 4.5 margins.

because its execution takes three orders of magnitude more clock cycles than the one of AES. Instead, we delimit the traces by finding peaks that are significantly larger than the others. In Figs. 4 and 5, there are distinguishable large peaks at the upper right. We use them as a starting point for synchronization and count clock cycles backwards from the starting point.

For each clock cycle to be synchronized, we find the peak corresponding to the rising edge of the randomized clock and set a threshold determined by the deviation from the mean. The deviation should be larger than 1.5σ, where the value of σ is calculated after excluding the starting peaks. In Figs. 4 and 5, the thresholds are marked by a red horizontal line.

For each byte of the message, the leakage is distributed among multiple clock cycles. We found that synchronizing a single clock cycle only is insufficient. To cover all potential leakage points for a given message byte with a margin, we synchronize 11 clock cycles.

Figs. 4 and 5 (middle) show the traces of K_u, K_r and K_{dr} after synchronization. The thresholds are indicated by red lines. Due to the varying overlap in time of the primary and dummy module's power consumption, traces of K_{dr} are harder to synchronize than the ones of K_r.

Table 1. Resources used for KYBER.CCAKEM.Decaps in different FPGA implementations; "CCC" stands for Clock Cycle Count.

Implementation	Protection type	Version	FPGA type	LUTs	FFs	BRAMs	CCC ($\times 10^3$)
This work	Randomized clock and duplication	Kyber768	Artix-7 XC7A100T	14,341	9,190	6	10
Xing and Li [7]	No protection	Kyber768	Artix-7 AC701	7,412	4,644	3	10
Kamucheka et al. [10]	Masking	Kyber512	Virtex-7 VC707	152,860	92,977∼ 743,816*	489.5	137.7
Huang et al. [14]	No protection	Kyber512	Artix-7 AC701	88,901	152,875∼ 1,223,000$^+$	202	68.8
Jati et al. [11]	Random delays, addr.&instr. shuffling	Kyber512	Artix-7 XC7A35T	7,151	3730	5.5	57.2

*The range of number of FFs based on the number of slices 92,977 reported in [10].
$^+$The range of number of FFs based on the number of slices 152,875 reported in in [14].

We also tried other synchronization techniques such as cross-correlation and wavelet transform. However, we got worse results, probably because they are not better at dealing with multiple short repeated patterns than the method of [31].

6.3 Leakage analysis

To analyze side-channel leakage, we use the Welch's t-test. For a given message byte $i \in \{0, 1, \ldots, 31\}$, we partition the profiling traces $\boldsymbol{T}_P$ into $\boldsymbol{T}_0$ and $\boldsymbol{T}_1$ as:

$$\boldsymbol{T}_0 = \{T \in \boldsymbol{T}_P \mid HW(m[i]) < 4\}$$
$$\boldsymbol{T}_1 = \{T \in \boldsymbol{T}_P \mid HW(m[i]) > 4\},$$

where $HW(m[i])$ is the Hamming weight of the ith byte of the message m encrypted in the ciphertext c which is given as input when the trace T is captured.

Figs. 4 and 5 (bottom) show the t-test scores for 200K synchronized traces from K_u, K_r and K_{dr}, respectively, for the message byte 24. An absolute value of t-test score higher than 4.5 at a point in time is considered a potential leakage. We can see that there are t-test scores higher than 4.5 in Fig. 4, however all scores in Fig. 5 are below 3.

6.4 Neural network training

Let $\mathbb{R}$ denote the set of real numbers and $\mathbb{I} := \{x \in \mathbb{R} \mid 0 \leq x \leq 1\}$.

To train a neural network $\mathcal{M} : \mathbb{R}^{|T|} \rightarrow \mathbb{I}^{256}$ to predict the value of a given message byte i, for any $i \in \{0, 1, \cdots, 31\}$, we use a labeled data set in which each trace $T \in \boldsymbol{T}_P$ is labeled by the value of the ith byte of the message m encrypted in the ciphertext c which is given as input when T is captured.

For all implementations K_u, K_r, and K_{dr}, we train MLPs with the same architecture and hyperparameters as in [8] except for the input size which in our case is 660 points.

7 Attack stage

At the attack stage, we use the sliced multi-bit error injection method introduced in [8]. The main idea is to inject all possible multi-bit errors e into the selected bytes of the message to be recovered. In this way, a non-differential message recovery side-channel attack is converted to a differential one.

To recover the message m from a properly generated ciphertext $c = (\mathbf{u}, v)$, for each $j \in \{0, 1, 2, 3\}$, we first construct 256 ciphertexts c_e decrypting to messages m_e defined by

$$m_e[i] = \begin{cases} m[i] \oplus e & \text{if } (i \bmod 4) = j, \\ m[i] & \text{otherwise,} \end{cases} \tag{1}$$

where $i \in \{0, 1, \cdots, 31\}$. The expression $m[i] \oplus e$ means that all bits of the byte $m[i]$ in which the 8-bit binary expansion of e has the value "1" are flipped. A single bit of m can be flipped by subtracting the center of the integer ring $\mathbb{Z}_q$ from the corresponding coefficient of v.

Then, we capture a set of 256×4 attack traces $\boldsymbol{T}_A$ during the decapsulation of the ciphertexts c_e. The message bytes $m[i]$, $i \in \{0, 1, \cdots, 31\}$, are recovered from the traces in $\boldsymbol{T}_A$ by giving the trace segments shown in Figs. 4 and 5 as input to the MLP models trained at the profiling stage.

For a trace $T_e \in \boldsymbol{T}_A$ captured with input ciphertext c_e, the model $\mathcal{M}$ outputs a score vector $S_{i,e} = \mathcal{M}(T_e)$ in which the value of the lth element, $S_{i,e}[l]$, is the probability that $m_e[i] = l$, for $l, e \in \{0, \ldots, 255\}$.

The most likely label for $m[i]$ among 256 candidates is determined as:

$$\tilde{l} = \underset{l \in \{0,1,\ldots,255\}}{\arg\max} \left(\prod_{e=0}^{255} S_{i,e}[l \oplus e]\right).$$

If $\tilde{l} = m[i]$, the classification is successful. The condition $\tilde{l} = m[i]$ can be verified by checking if the rank[3] of $m[i]$ is zero.

8 Experimental Results

To evaluate the presented method, we performed side-channel attacks on: (1) the implementation K_{dr} protected by a duplication and randomized clock, (2) the implementation K_r protected by a randomized clock only, and (3) the unprotected implementation K_u from [7].

This section presents the security analysis and overhead analysis.

8.1 Security Analysis

First, we trained one MLP model per implementation, K_u, K_r and K_{dr}, on the profiling sets of traces $\boldsymbol{T}_P$ captured from the FPGA and synchronized as

[3] A *rank* of a byte b is the number of other bytes which have a probability higher than b in the score vector.

Table 2. The results of message recovery attacks.

Implementation	Byte rank for 10 messages										Average
	0	1	2	3	4	5	6	7	8	9	
$\times 2$ K_u	0	2	0	0	0	0	0	0	0	0	0.2
$\times 30$ K_u	0	0	0	0	0	0	0	0	0	0	0
$\times 30$ K_r	0	73	1	0	18	0	13	50	91	13	25.9
$\times 30$ K_{dr}	90	204	16	164	59	8	104	10	38	95	78.8

described in Section 6. A new synchronization is required for each message byte. All the experiments in this section are performed for byte 24 for two reasons. The first reason is that previous attack on the same KYBER implementation [8] has shown it has a higher leakage than the other bytes. The second reason is that Byte 24 is the easiest to synchronize. Therefore we select to analyse the easiest byte to attack to make a proper security evaluation of our countermeasure.

For each implementation, we captured $|\boldsymbol{T}_P| = 200$K traces. After the synchronization, the size of $\boldsymbol{T}_P$ decreased to 194K and 180K, for K_r and K_{dr}, respectively, due to the removal of traces which could not be synchronized.

Then, we captured the attack traces $\boldsymbol{T}_A$. We generated 10 messages at random and encrypted them using CPAPKE.Enc. For each of 10 resulting ciphertexts c, we constructed 256 ciphertexts c_e decrypting to messages m_e defined by the equation (1) for $j = 0$ ($24 \bmod 4 = 0$). Then, we captured traces for these 256 ciphertexts with 10 repetitions from each implementation K_u, K_r and K_{dr}. After the synchronization, the number of repeated groups reduced to 8 and 7 for K_r and K_{dr}, respectively.

Table 2 summarizes the results of message recovery attacks. It lists the byte rank for each of the 10 test messages. The first two lines of Table 2 represent the attacks on the unprotected implementation K_u from [7]. We included both cases, oversampling factor $\times 2$ and $\times 30$, to show that a higher oversampling factor[4] can improve the results. One can see that the byte rank of message 1 is reduced from 2 to 0.

The last two lines of Table 2 represent the attacks on the protected implementations K_r and K_{dr}. For K_r, in 4 out of 10 cases the attacker can recover the message byte by enumerating two candidates (with ranks 0 and 1). Assuming all 32 message bytes have the same rank, this gives the enumeration complexity of 2^{32}, which is feasible. On the other hand, for K_{dr}, the ranks are all messages, something too high to enumerate.

8.2 Overhead analysis

Table 1 lists resources used for implementing the decapsulation algorithm of CRYSTALS-Kyber in the FPGA implementations available at present.

[4] A signal is said to be oversampled by a factor of N if it is sampled at N times the Nyquist rate (two times the signal frequency).

The duplication and clock randomization countermeasure has a close to 100% overhead in area. However, it does not increase the number of clock cycles, although an average frequency of K_{dr} will be lower than the frequency of K_u due to the randomized clock. Another consequence of a randomized clock is non-constant throughput and latency (note that in an infinite time window, the average throughput is constant). Masking increases both the area and the number of clock cycles. The masking method of [10] adds 72% of logic elements (LUTs), about 100% of memory elements (BRAMs and FFs), and doubles the number of clock cycles. Note, however, that the overhead of masking is implementation-dependent. The highly optimized implementation of CRYSTALS-Kyber from [7] is likely to have a higher overhead in the masked version.

The area overhead of the random delays and address and instruction shuffling countermeasure of [11] is only 5%. However, random delay insertion increases the clock cycle count. Furthermore, it is necessary to evaluate [11] using measurements captured at a high oversampling rate to draw safe conclusions about its security.

9 Conclusion

We implemented and evaluated a method for protecting FPGA implementations of CRYSTALS-Kyber from side-channel attacks based on duplication and clock randomization.

The protected implementations of Kyber-768 as well as the data used in the experiments are available at: `https://github.com/YanningJi/SCA-Protected-Kyber-FPGA`. They are hoped to be useful as a benchmark in the follow up evaluations.

Acknowledgements

This work was supported in part by the research grant 2021-02426 from Vinnova, the research grant 2020-11632 from the Swedish Civil Contingencies Agency, and the research grant 2018-04482 from the Swedish Research Council.

References

1. P. W. Shor, "Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer," *SIAM review*, vol. 41, no. 2, pp. 303–332, 1999.
2. R. Avanzi, J. Bos, E. K. Léo Ducas, T. Lepoint, V. Lyubashevsky, J. M. Schanck, P. Schwabe, G. Seiler, and D. Stehlé, "CRYSTALS-Kyber algorithm specifications and supporting documentation," 2021. `pq-crystals.org/kyber/data/kyber-specification-round3-20210131.pdf`.
3. D. Moody, "Status Report on the Third Round of the NIST Post-Quantum Cryptography Standardization Process," *Nistir 8309*, pp. 1–27, 2022.
4. "Announcing the commercial national security algorithm suite 2.0," *National Security Agency, U.S Department of Defense*, Sep 2022.

5. P. Kocher, J. Jaffe, and B. Jun, "Differential power analysis," in *Annual international cryptology conference*, pp. 388–397, Springer, 1999.
6. L. Backlund, K. Ngo, J. Gartner, and E. Dubrova, "Secret key recovery attacks on masked and shuffled implementations of CRYSTALS-Kyber and Saber." Cryptology ePrint Archive, Paper 2022/1692, 2022. https://eprint.iacr.org/2022/1692.
7. Y. Xing and S. Li, "A compact hardware implementation of CCA-secure key exchange mechanism CRYSTALS-KYBER on FPGA," *IACR Transactions on Cryptographic Hardware and Embedded Systems*, pp. 328–356, 2021.
8. Y. Ji, R. Wang, K. Ngo, and E. Dubrova, "A side-channel attack on a hardware implementation of CRYSTALS-Kyber," in *2023 IEEE European Test Symposium (ETS'23)*, 2023.
9. R. C. Rodriguez, F. Bruguier, E. Valea, and P. Benoit, "Correlation electromagnetic analysis on an FPGA implementation of CRYSTALS-Kyber." Cryptology ePrint Archive, Paper 2022/1361, 2022. https://eprint.iacr.org/2022/1361.
10. T. Kamucheka, A. Nelson, D. Andrews, and M. Huang, "A masked pure-hardware implementation of Kyber cryptographic algorithm," in *NIST Fourth PQC Standardization Conference*, 2022. https://eprint.iacr.org/2022/1547.
11. A. Jati, N. Gupta, A. Chattopadhyay, and S. K. Sanadhya, "A configurable CRYSTALS-Kyber hardware implementation with side-channel protection." Cryptology ePrint Archive, Paper 2021/1189, 2021. https://eprint.iacr.org/2021/1189.
12. M. Moraitis, Y. Ji, M. Brisfors, E. Dubrova, N. Lindskog, and H. Englund, "Securing CRYSTALS-Kyber in FPGA using duplication and clock randomization," *IEEE Design Test*, pp. 1–1, 2023.
13. M. Moraitis, M. Brisfors, E. Dubrova, N. Lindskog, and H. Englund, "A protected hardware implementation of AES using clock randomization and duplication," in *2023 IEEE International Symposium on Circuits and Systems (ISCAS), Monterey, California, USA*, 2023.
14. Y. Huang, M. Huang, Z. Lei, and J. Wu, "A pure hardware implementation of CRYSTALS-KYBER PQC algorithm through resource reuse," *IEICE Electronics Express*, vol. 17, no. 17, pp. 20200234–20200234, 2020.
15. L. Botros, M. J. Kannwischer, and P. Schwabe, "Memory-efficient high-speed implementation of kyber on cortex-m4," in *Progress in Cryptology – AFRICACRYPT 2019* (J. Buchmann, A. Nitaj, and T. Rachidi, eds.), (Cham), pp. 209–228, Springer International Publishing, 2019.
16. T. Kamucheka, M. Fahr, T. Teague, A. Nelson, D. Andrews, and M. Huang, "Power-based side channel attack analysis on PQC algorithms," 2021. https://eprint.iacr.org/2021/1021.
17. P. C. Kocher, J. Jaffe, and B. Jun, "Using unpredictable information to minimize leakage from smartcards and other cryptosystems." US Patent 6,327,661.
18. M. Bucci, R. Luzzi, M. Guglielmo, and A. Trifiletti, "A countermeasure against differential power analysis based on random delay insertion," in *2005 IEEE International Symposium on Circuits and Systems (ISCAS)*, pp. 3547–3550 Vol. 4, 2005.
19. Y. Lu, M. P. O'Neill, and J. V. McCanny, "FPGA Implementation and Analysis of Random Delay Insertion Countermeasure against DPA," in *2008 International Conference on Field-Programmable Technology*, pp. 201–208, 2008.
20. Y. Zafar and D. Har, "A Novel Countermeasure Enhancing Side Channel Immunity in FPGAs," in *2008 International Conference on Advances in Electronics and Micro-electronics*, pp. 132–137, 2008.

21. Y. Zafar, J. Park, and D. Har, "Random clocking induced DPA attack immunity in FPGAs," in *2010 IEEE International Conference on Industrial Technology*, pp. 1068–1070, 2010.
22. K. H. Boey, P. Hodgers, Y. Lu, M. O'Neill, and R. Woods, "Security of AES Sbox designs to power analysis," in *2010 17th IEEE International Conference on Electronics, Circuits and Systems*, pp. 1232–1235, 2010.
23. K. H. Boey, Y. Lu, M. O'Neill, and R. Woods, "Random clock against differential power analysis," in *2010 IEEE Asia Pacific Conference on Circuits and Systems*, pp. 756–759, 2010.
24. T. Güneysu and A. Moradi, "Generic side-channel countermeasures for reconfigurable devices," in *International Workshop on Cryptographic Hardware and Embedded Systems*, pp. 33–48, Springer, 2011.
25. P. Ravi, S. Bhasin, J. Breier, and A. Chattopadhyay, "PPAP and iPPAP: PLL-based protection against physical attacks," in *2018 IEEE Computer Society Annual Symposium on VLSI (ISVLSI)*, pp. 620–625, IEEE, 2018.
26. J.-S. Coron and I. Kizhvatov, "An efficient method for random delay generation in embedded software," in *International Workshop on Cryptographic Hardware and Embedded Systems*, pp. 156–170, Springer, 2009.
27. J.-S. Coron and I. Kizhvatov, "Analysis and improvement of the random delay countermeasure of CHES 2009," in *International Workshop on Cryptographic Hardware and Embedded Systems*, pp. 95–109, Springer, 2010.
28. D. Jayasinghe, A. Ignjatovic, and S. Parameswaran, "RFTC: Runtime frequency tuning countermeasure using FPGA dynamic reconfiguration to mitigate power analysis attacks," in *2019 56th ACM/IEEE Design Automation Conference (DAC)*, pp. 1–6, IEEE, 2019.
29. B. Hettwer, K. Das, S. Leger, S. Gehrer, and T. Güneysu, "Lightweight Side-Channel Protection using Dynamic Clock Randomization," in *2020 30th International Conference on Field-Programmable Logic and Applications (FPL)*, pp. 200–207, 2020.
30. D. Jayasinghe, A. Ignjatovic, and S. Parameswaran, "Uclod: Small clock delays to mitigate remote power analysis attacks," *IEEE Access*, vol. 9, pp. 108411–108425, 2021.
31. M. Brisfors, M. Moraitis, and E. Dubrova, "Do not rely on clock randomization: A side-channel attack on a protected hardware implementation of AES," in *Proc. of the 15th International Symposium on Foundations Practice of Security (FPS – 2022)*, Springer.
32. S. Chari, C. S. Jutla, J. R. Rao, and P. Rohatgi, "Towards sound approaches to counteract power-analysis attacks," in *Advances in Cryptology - CRYPTO '99*, vol. 1666, pp. 398–412, Springer, 1999.
33. E. Dubrova, K. Ngo, and J. Gartner, "Breaking a fifth-order masked implementation of CRYSTALS-Kyber by copy-paste." Cryptology ePrint Archive, Paper 2022/, 2022. `https://eprint.iacr.org/2022/`.
34. S. Takarabt, S. Guilley, Y. Souissi, K. Karray, L. Sauvage, and Y. Mathieu, "Formal evaluation and construction of glitch-resistant masked functions," in *IEEE International Symposium on Hardware Oriented Security and Trust (HOST'2021)*, pp. 304–313, 2021.
35. Veyrat-Charvillon *et al.*, "Shuffling against side-channel attacks: A comprehensive study with cautionary note," in *Advances in Cryptology – ASIACRYPT 2012*, pp. 740–757, Springer Berlin Heidelberg, 2012.

36. P. Ravi, S. Bhasin, S. S. Roy, and A. Chattopadhyay, "On exploiting message leakage in (few) NIST PQC candidates for practical message recovery attacks," *IEEE Transactions on Information Forensics and Security*, 2021.
37. E. Fujisaki and T. Okamoto, "Secure integration of asymmetric and symmetric encryption schemes," in *Annual international cryptology conference*, pp. 537–554, Springer, 1999.
38. NewAE Technology Inc., "Chipwhisperer." `https://newae.com/tools/chipwhisperer`.